

Universidade de Vigo

Memoria Proyecto - Sistemas Inteligentes

Warehouse

Grupo SI 8-2

Luis Garbayo Fernández
Yerai Fernández Rodríguez
Alberto Andrés Faublak Álvarez
Vincenzo Gagliano
Eva Barreiro Ferreira

2025–2026

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Índice general

1. Introducción	1
2. Antecedentes y contexto	3
2.1. Herramientas y tecnologías existentes	3
2.1.1. El modelo BDI y AgentSpeak	3
2.1.2. La plataforma Jason	4
2.1.3. La metodología Prometheus	4
2.1.4. El patrón Modelo-Vista-Controlador	4
2.1.5. Navegación distribuida con heurística Manhattan	5
2.2. Conclusión	6
3. Objetivos	7
3.1. Objetivo General	7
3.2. Objetivos específicos	7
3.2.1. Primera semana: objetivos iniciales	7
3.2.2. Segunda semana: objetivos incrementales	8
3.2.3. Tercera semana: zona de salida y coordinación autónoma	9
3.2.4. Cuarta semana: ciclo de salida y control de <i>deadlines</i>	9
3.2.5. Quinta semana: monitorización y control temporal	10
4. Diseño del SMA	11
4.1. Diagrama de entorno	11

4.2. Organización del sistema	13
4.2.1. Redefinición de roles y responsabilidades	13
4.2.2. Nuevos componentes de la organización	14
4.2.3. Evolución del modelo de comunicación	14
4.3. Diagrama de objetivos ideal	14
4.4. Diagrama de objetivos	15
4.5. Diagrama de interacción	17
4.6. Agentes y tareas	18
4.6.1. Agente Robot (light, medium, heavy, heavy2)	18
4.6.2. Agente Scheduler	21
4.6.3. Agente Supervisor: Arbitraje y Auditoría Crítica	23
4.6.4. Agente Transport: Interfaz de Salida y Seguridad	24
5. Arquitectura: Patrón MVC (Modelo-Vista-Controlador)	25
5.1. El Modelo: datos y estado del almacén	25
5.2. La Vista: interfaz gráfica	26
5.3. El Controlador: artefacto y agentes	26
6. Implementación de la solución	28
6.1. El Entorno Java: Interfaz de Percepción y Control	28
6.1.1. Modelo de la cuadrícula y zonas	28
6.1.2. Gestión de Contenedores y Ciclo de Vida	29
6.1.3. API de Acciones y Entorno	29
6.2. Navegación distribuida en agentes ASL	30
6.2.1. Por qué se eliminó el BFS	30
6.2.2. Heurística <i>greedy</i> de distancia Manhattan	30

6.2.3. Sistema de pasillos verticales	30
6.2.4. Gestión de colisiones con <i>backoff</i> escalonado	31
6.2.5. nav_limit y nav_target	31
6.3. Coordinación Distribuida y Planificación	31
6.3.1. Planificación distribuida como alternativa al planificador central	31
6.3.2. Reclamo atómico de contenedores	32
6.3.3. Selección autónoma de estantería	33
6.3.4. blocked_type: inhibición coordinada de reclamaciones	33
6.3.5. Mutex de zona y optimización de check_queue	34
6.4. El Agente Scheduler	34
6.4.1. Activación del ciclo de salida	34
6.4.2. Diseño asimétrico de fases	35
6.4.3. Desbloqueo anticipado al recuperar espacio	35
6.5. Los Agentes Robot	36
6.5.1. Arquitectura compartida mediante common.asl	36
6.5.2. Capacidades diferenciadas y ciclo inbound	36
6.5.3. Ciclo de salida autónomo	36
6.5.4. nav_failed y shelf_wait	37
6.6. El Agente Supervisor	37
6.6.1. Detección de saturación	37
6.6.2. Seguimiento de deadlines e incumplimientos	37
6.7. El Agente Transport	38
6.8. Robustez y Gestión de Fallos	38
6.9. Logging de eventos obligatorios	39

7. Pruebas y Resultados	40
7.1. Escenario base: operación nominal	40
7.2. Escenario de saturación del almacén	41
7.3. Ciclo de salida urgente	41
7.4. Ciclo de salida no urgente	42
7.5. Zona de expansión y desbordamiento de estantería	43
7.6. Outbound lleno: recogida reactiva por Transport	43
8. Conclusiones y trabajo futuro	45

Índice de tablas

6.1. Resumen de eventos obligatorios monitorizados.	39
---	----

Índice de figuras

4.1. Diagrama de entorno actualizado para la coordinación distribuida.	13
4.2. Diagrama de objetivos ideal con flujos inbound y outbound independientes.	15
4.3. Diagrama de objetivos detallando la descomposición de tareas por agente.	17
4.4. Diagrama de interacción detallando la coordinación distribuida y los protocolos de mutex.	18
4.5. Arquitectura BDI del agente robot autónomo con capacidades de decisión distribuida.	19
4.6. Diagrama de tareas del robot detallando la lógica de reclamo y selección autónoma.	21
4.7. Arquitectura del agente Scheduler centrada en la gestión de ciclos y deadlines.	22

1. Introducción

Gestionar un almacén logístico de forma autónoma supone un reto de coordinación. No basta con que cada robot sepa moverse; el sistema debe lidiar con agentes físicos de capacidades heterogéneas que convergen en un mismo espacio. Aquí, la dificultad real se refleja en la interacción, es decir, en los posibles cuellos de botella, la distribución de tareas en tiempo real y la absorción de fallos mecánicos sin que un humano tome las riendas. El éxito no depende del movimiento individual, sino de una inteligencia colectiva capaz de garantizar el flujo constante de mercancía.

Los Sistemas Multiagente (SMA) constituyen un paradigma especialmente adecuado para abordar este tipo de problemas. Su capacidad para modelar entidades autónomas que perciben el entorno, razonan sobre sus objetivos y coordinan su comportamiento mediante paso de mensajes permite descomponer la complejidad global del sistema en agentes individuales con responsabilidades bien definidas. En particular, el modelo de agente BDI (*Beliefs, Desires, Intentions*) proporciona una base formal sólida para especificar el comportamiento reactivo y deliberativo de cada entidad, facilitando tanto el diseño como el análisis del sistema resultante.

Este proyecto parte de una base de código proporcionada por la asignatura y la extiende sustancialmente. La intervención ha abarcado desde la lógica de los agentes hasta el entorno Java y su interfaz visual, logrando un ecosistema logístico autosuficiente. El proceso de ingeniería de los agentes y sus interacciones se apoya en la metodología Prometheus, que estructura la arquitectura mediante diagramas de entorno, organización, objetivos, interacción y tareas, asegurando una transición sólida hacia la implementación final en Jason. En paralelo, la robustez del *software* se garantiza mediante un patrón Modelo-Vista-Controlador (MVC), que blindará la separación entre la lógica de negocio y la gestión física del entorno.

El sistema resultante de la primera iteración coordina de forma autónoma el transporte de contenedores desde la zona de entrada hasta las estanterías, apoyándose en tres tipologías de robots con capacidades diferenciadas (*light, medium* y *heavy*). Esta operativa está gobernada por un agente planificador (*scheduler*), que centraliza la toma de decisiones estratégicas, y un agente supervisor (*supervisor*) encargado de auditar la integridad y el estado global del almacén en tiempo real.

Tras la segunda y última iteración se expande el alcance del sistema mediante tres ejes fundamentales que refuerzan su autonomía. En primer lugar, se implementa el ciclo completo de salida de

mercancía, donde el sistema debe coordinarse con un nuevo agente externo (*transport*) y gestionar *deadlines* temporales para priorizar envíos críticos bajo la auditoría del agente *supervisor*. Por otro lado, el modelo evoluciona hacia una coordinación distribuida: los robots (incluyendo una nueva unidad de tipo *heavy*) asumen ahora la responsabilidad de reclamar y gestionar contenedores de forma proactiva, eliminando la dependencia del *scheduler* en la asignación de tareas individuales. Finalmente, se ha consolidado un “entorno delgado” (*thin environment*), donde el entorno Java se despoja de su antigua lógica de navegación para limitarse a funciones de percepción, trasladando todo el peso de la deliberación espacial y el razonamiento táctico al núcleo de los propios agentes.

2. Antecedentes y contexto

Los sistemas multiagente llevan décadas evolucionando desde sus bases teóricas hasta plataformas concretas usadas tanto en investigación como en industria. Lo que nació como un formalismo para describir el comportamiento de entidades computacionales es hoy un ecosistema capaz de orquestar agentes heterogéneas en entornos físicos de alta complejidad. En el núcleo de esta evolución emerge el paradigma BDI, proporcionando un marco cognitivo que permite a los agentes procesar su conocimiento del entorno, priorizar objetivos y adquirir compromisos de acción coherentes. Este capítulo contextualiza el proyecto detallando las herramientas y paradigmas que lo sustentan, revisando las decisiones de diseño que dotan de solidez y justifican la implementación final.

2.1. Herramientas y tecnologías existentes

2.1.1. El modelo BDI y AgentSpeak

Cuando se trata de diseñar agentes deliberativos, el modelo BDI (*Beliefs, Desires, Intentions*) se ha consolidado como el estándar por excelencia desde que Rao y Georgeff lo formalizaran en 1995 [1]. La clave de este enfoque, y lo que lo aleja de los sistemas reactivos simples, es que dota al agente de una “psicología” propia: sus creencias (*beliefs*) actúan como un mapa del mundo, sus deseos (*desires*) marcan las metas finales y sus intenciones (*intentions*) definen el plan de acción al que se compromete. Para la logística de este almacén, dicha arquitectura es fundamental; nos permite gestionar objetivos a largo plazo sin perder la capacidad de reacción ante un obstáculo imprevisto en el pasillo.

Para programar esta lógica se usó AgentSpeak [2]. No es un lenguaje convencional, sino que está basado en planes que se disparan según eventos. Cada plan tiene su contexto de aplicabilidad, esto es una serie de condiciones que deben cumplirse, y un cuerpo de acciones. Esta estructura es la que dicta cómo el robot selecciona una respuesta ante cada evento y cómo debe actuar si algo sale mal mediante los planes de contingencia.

2.1.2. La plataforma Jason

Jason [3] [4] es la implementación de referencia de AgentSpeak. Proporciona un intérprete del ciclo de razonamiento BDI, soporte para comunicación entre agentes mediante paso de mensajes asíncronos, y una API Java que permite conectar los agentes con entornos externos implementados en dicho lenguaje. Esta última característica es especialmente relevante para el presente proyecto: la clase `jason.environment.Environment` actúa como puente entre la lógica de los agentes `.asl` y el artefacto Java `WarehouseArtifact`, que implementa la física del almacén.

Jason se ejecuta sobre la máquina virtual de Java, lo que garantiza portabilidad, y dispone de herramientas de depuración integradas como el *Agent Mind Inspector*, que permite inspeccionar en tiempo real la base de creencias, las intenciones activas y el ciclo de razonamiento de cada agente durante la ejecución, y ha sido de gran ayuda a la hora de la implementación del proyecto y de posibles errores detectados en ella.

2.1.3. La metodología Prometheus

Para que el desarrollo del sistema se siguió la metodología Prometheus [6]. A diferencia de la ingeniería de software tradicional, esta está pensada exclusivamente para agentes, ofreciendo una ruta clara que va desde la idea inicial hasta la implementación. En este proyecto, Prometheus ha sido la hoja de ruta para definir a nuestros agentes: mapeando sus percepciones, qué acciones pueden ejecutar y cómo se comunican entre ellos. Es un paso previo al código en Jason que, como veremos en los diagramas de capítulos posteriores, permite que la estructura final sea mucho más robusta y no una simple improvisación.

2.1.4. El patrón Modelo-Vista-Controlador

En cuanto a la infraestructura Java, se organizó el código bajo el patrón MVC (*Model-View-Controller*). El objetivo principal era no mezclar la lógica de los robots con la interfaz gráfica. Así, el modelo (con clases como `Container` o `Robot`) se encarga exclusivamente de guardar el estado del almacén, mientras que la vista (`WarehouseView`) solo se preocupa de dibujar todo en pantalla con Swing. El nexo de unión es el controlador (`WarehouseArtifact`), que es quien recibe las órdenes de los agentes, actualiza el mapa y avisa a la interfaz de que algo ha cambiado. Esta separación es lo que permite retocar la lógica de negocio sin miedo a romper la visualización, y viceversa.

2.1.5. Navegación distribuida con heurística Manhattan

La navegación de los robots en el almacén se resolvía en la primera iteración mediante búsqueda en anchura (BFS) implementada en el entorno Java. Este enfoque presentaba dos problemas. El primero es de arquitectura: el entorno calculaba la ruta completa y decidía cada paso de movimiento, un razonamiento que en el modelo BDI corresponde al agente. El segundo es de eficiencia: en el peor caso BFS exploraba hasta los 300 nodos del grid antes de devolver el primer paso, repitiendo ese cálculo en cada movimiento.

Para este diseño se utilizó como base el patrón del proyecto *domestic-robot* proporcionado como ejemplo por Jason. Se trata de un plan recursivo que llama a una acción de movimiento y se vuelve a aplicar hasta llegar al destino. Pero en el *warehouse* se ha ido un paso más allá: mientras que en el ejercicio base `move_towards(P)` calcula el paso en Java, aquí el entorno solo ofrece la acción `move_step(X,Y)`. Esta acción mueve al robot exactamente una celda física, dejando que sea el agente quien decida el rumbo mediante el objetivo `!do_step`.

Con este cambio, el entorno Java se limita a ser un proveedor de información. Acciones como `move_to_shelf(Id)`, `move_to_container(Id)` o `move_to_outbound` ahora notifican al agente las coordenadas del destino mediante la percepción `nav_target(TX, TY)`. Pero es el caso de `move_to_expansion` el que mejor refleja esta migración: mientras que antes Java calculaba y entregaba la mejor celda ya filtrada, ahora el entorno se limita a volcar todas las opciones disponibles mediante percepciones de todas las celdas libres con su distancia Manhattan ya calculada `expansion_free_cell(D, X, Y)`. Es el agente quien utiliza un `.sort` para elegir la mínima distancia. De este modo, la elección del destino deja de ser una “caja negra” en Java y pasa a ser una decisión deliberativa del propio robot.

En cuanto a la lógica de rutas, el agente decide su próximo movimiento usando una heurística *greedy* de Manhattan. La regla es sencilla: avanzar siempre por el eje donde la distancia sea mayor ($|\Delta X| \geq |\Delta Y| \rightarrow$ eje X primero). Esto constituye una solución eficiente para un tablero ortogonal. Sin embargo, un algoritmo puramente *greedy* se chocaría contra las filas de estanterías al intentar atravesarlas. Para evitarlo, se diseñó un sistema de pasillos verticales que actúan como “autopistas” en $x = 9$ (pasillo izquierdo) y $x = 19$ (pasillo derecho).

El patrón resultante sigue una jerarquía de objetivos: posición actual $\rightarrow$ pasillo vertical $\rightarrow$ fila destino $\rightarrow$ destino final. Bajo esta lógica, en el plan `!navigate` se programaron reglas de *routing* específicas: por ejemplo, los destinos en la zona izquierda ($TX < 9$) obligan al robot a buscar primero el pasillo de $x = 9$, mientras que los cambios de fila dentro del área de estanterías se resuelven dirigiéndose al pasillo más cercano según la columna actual ($x = 9$ si $X \leq 14$, $x = 19$ en caso

contrario). Esta estructura de *waypoints* asegura que el robot mantenga una trayectoria fluida y nunca se quede encajonado.

Además, al moverse varios robots a la vez, las colisiones son inevitables. Por ello, se programó un sistema de gestión de errores en el plan `!step_with_retry` basado en un contador de bloqueos (*Block Count*, BC):

- **BC = 0:** El robot intenta el paso de forma normal y optimista.
- **BC entre 1 y 2:** El bloqueo se considera transitorio. El robot espera un tiempo aleatorio (300-1500 ms) para romper la sincronía con otros posibles robots y reintenta.
- **BC entre 2 y 4:** El agente aplica un *path-backoff*, ejecutando un paso lateral perpendicular para intentar sortear el obstáculo.
- **BC entre 4 y 6:** Se activa una espera larga (1500-3000 ms) para permitir que congestiones severas en los pasillos se despejen.
- **BC $\geq$ 6:** El robot notifica `path_blocked` al *supervisor* y aborta el plan. Esto resetea el estado del robot y libera el contenedor para que pueda ser gestionado por otro agente, evitando que el sistema se bloquee permanentemente.

Por último, para que ningún robot se quede dando vueltas infinitamente por un error de lógica, se añadió un límite de seguridad de 300 pasos (`nav_limit`). Si se supera dicho límite, el agente lanza `nav_timeout` y detiene su intención actual.

2.2. Conclusión

BDI, Jason y Prometheus son opciones bien establecidas y documentadas para este tipo de sistemas; su uso en el proyecto no es arbitrario sino que responde directamente a los requisitos de la asignatura y a la naturaleza del problema que se quiere resolver.

3. Objetivos

3.1. Objetivo General

El propósito fundamental de este proyecto es el diseño y desarrollo de un Sistema Multiagente (SMA) capaz de gobernar, de manera íntegramente autónoma, la cadena logística completa de un almacén simulado. El sistema integra agentes BDI con roles especializados para resolver el flujo integral de mercancía: desde la recepción y clasificación inteligente de contenedores (*inbound*), hasta su gestión en inventario y posterior extracción hacia la zona de salida (*outbound*). Para alcanzar una operatividad eficiente, el diseño se apoya en una coordinación distribuida, donde los robots actúan como entidades proactivas que reclaman tareas directamente del entorno según sus capacidades físicas, eliminando la necesidad de un despachador centralizado para la asignación de trabajos. Asimismo, el sistema está dimensionado para cumplir con restricciones temporales críticas (*deadlines*) en los ciclos de salida, garantizando una respuesta ágil ante la saturación del inventario. Finalmente, el SMA incorpora mecanismos de supervisión y robustez para auditar el estado global del almacén, permitiendo la detección de errores y la autorregulación del sistema sin intervención humana directa.

3.2. Objetivos específicos

3.2.1. Primera semana: objetivos iniciales

- **El *scheduler* detecta nuevos contenedores.** El agente planificador debe ser capaz de percibir en tiempo real la llegada de nuevos contenedores al almacén e iniciar su procesamiento de forma autónoma.
- **El *scheduler* asigna el contenedor a un robot.** El planificador debe seleccionar, para cada contenedor, el robot más adecuado en función de sus características, garantizando una distribución de tareas coherente con las capacidades de cada robot.
- **El *scheduler* asigna estantería al contenedor.** Cada contenedor debe tener una estantería destino reservada antes de ser transportado, evitando conflictos de almacenamiento.

- **El robot realiza la tarea asignada.** Los robots deben ejecutar de forma autónoma las tareas que les asigne el *scheduler*, sin intervención externa durante el proceso.
- **El robot recoge el contenedor y lo lleva a la estantería.** El ciclo completo, desde la recogida del contenedor hasta su depósito en la estantería, debe ser ejecutado íntegramente por el robot.
- **El robot encuentra el camino a la estantería.** Los robots deben ser capaces de calcular rutas válidas por el almacén para alcanzar tanto los contenedores como las estanterías destino.
- **Todos los robots llevan contenedores a estanterías.** Los tres tipos de robot (ligero, mediano y pesado) deben ser operativos y capaces de completar transportes de forma simultánea e independiente.
- **Los robots manejan errores de asignación.** Ante situaciones de error durante la ejecución de una tarea, los robots deben gestionarlas adecuadamente y notificar al resto del sistema para permitir la recuperación.
- **El supervisor monitoriza contenedores recibidos, almacenados y errores.** El agente supervisor debe mantener un registro actualizado del estado de todos los contenedores que circulan por el almacén.
- **El supervisor calcula estadísticas.** El supervisor debe producir periódicamente informes cuantitativos sobre el rendimiento global del sistema.

3.2.2. Segunda semana: objetivos incrementales

- **El *scheduler* recepciona contenedores.** El planificador debe suscribirse a las notificaciones del entorno para detectar cada nuevo contenedor en el momento de su llegada.
- **El *scheduler* clasifica el contenedor por peso, tamaño y tipo.** Cada contenedor debe ser categorizado según tres dimensiones independientes: peso, dimensiones y tipo de carga, para guiar las decisiones de asignación posteriores.
- **El *scheduler* sitúa los contenedores en puntos distintos.** Los contenedores deben distribuirse entre varias posiciones de entrada del almacén, evitando que todos se acumulen en el mismo punto.
- **El *scheduler* asigna la estantería en la que almacenar el contenedor clasificado.** La elección de estantería debe tener en cuenta la clasificación previa del contenedor, agrupando cada tipo en el área de almacenamiento correspondiente.

- El **scheduler** mantiene la trazabilidad robot–contenedor–estantería. El planificador debe registrar en todo momento qué robot es responsable de cada contenedor y a qué estantería está destinado.
- El **supervisor** monitoriza el estado de los robots. El supervisor debe conocer en todo momento si cada robot está activo o en espera, reflejando los cambios de estado a medida que se producen.

3.2.3. Tercera semana: zona de salida y coordinación autónoma

- **Reorganización del layout del almacén.** Se deben definir y delimitar nuevas áreas funcionales: salida (celdas 0-2), clasificación (celdas 3-4) y entrada (celdas 5-7), permitiendo un flujo de trabajo adaptado al ciclo de salida.
- **Implementación de la coordinación distribuida y escalabilidad.** Los robots deben reclamar contenedores proactivamente basándose en creencias locales, eliminando las asignaciones del *scheduler*. Se incorpora además una segunda unidad de robot pesado para reforzar la capacidad del sistema.
- **Categorización funcional de estanterías.** Se debe implementar la lógica de almacenamiento por tipo: estanterías S1, S5 y S8 para contenedores urgentes, y el resto para carga estándar y frágil.
- **Detección de saturación y comunicación de estados.** El supervisor debe identificar la falta de espacio por tipo y notificar al *scheduler*, quien activará el ciclo de salida correspondiente y bloqueará la entrada de nuevos contenedores de ese tipo.
- **Estandarización del logging obligatorio.** El sistema debe registrar formalmente los eventos de saturación detectados por el supervisor (`no_space_detected`) y el inicio de las fases de salida por parte del *scheduler* (`output_phase_started`).

3.2.4. Cuarta semana: ciclo de salida y control de *deadlines*

- **Gestión de fases y control temporal del ciclo.** El *scheduler* debe activar el ciclo de salida tras la recepción del aviso de saturación (T_0), gestionando dos plazos secuenciales: un *deadline* corto para carga urgente y uno largo para el resto de contenedores.
- **Optimización y justificación de márgenes temporales.** Se deben establecer y documentar valores de ΔT realistas (basados en pruebas de rendimiento), garantizando que solo un *dead-*

line esté activo en cada instante para evitar conflictos operativos.

- **Autonomía proactiva bajo restricciones temporales.** Los robots deben identificar y evacuar de forma autónoma únicamente los contenedores permitidos por el *deadline* activo, tomando decisiones de transporte sin recibir asignaciones explícitas del *scheduler*.
- **Coordinación con el agente *Transport*.** Se debe integrar el agente de transporte para que, al finalizar cada fase, recoja los contenedores depositados en la zona de salida, asegurando que el sistema no se bloquee por falta de espacio en el *outbound*.
- **Trazabilidad y registro de eventos obligatorios.** El sistema debe emitir logs específicos para el inicio y fin de cada *deadline* (*deadline_started/ended*) y para cada entrega efectiva realizada por los robots (*container_delivered*), permitiendo verificar el comportamiento del sistema bajo carga.

3.2.5. Quinta semana: monitorización y control temporal

- **Detección pasiva de incumplimientos temporales.** El supervisor debe auditar el cumplimiento de los *deadlines* sin interferir en la ejecución del sistema, garantizando que no se cancelen tareas en curso ni se modifique el proceso de decisión de los robots.
- **Implementación de monitorización periódica y criterio de fallo.** Se debe establecer un ciclo de revisión recurrente (cada 5 segundos) para identificar contenedores que, habiendo superado el tiempo límite de *deadline* no han sido depositados en la zona de salida.
- **Registro individualizado de errores de *deadline*.** El sistema debe ser capaz de registrar un incumplimiento específico por cada contenedor pendiente, manteniendo la trazabilidad exacta de qué elementos no cumplieron con los requisitos logísticos.
- **Robustez mediante mecanismos de auditoría dual.** Se deben implementar mecanismos de seguridad (monitores y *handlers*) que aseguren la detección del fallo incluso si el ciclo de revisión coincide con el fin exacto de la fase, evitando duplicidades en el registro.
- **Trazabilidad mediante *logging* obligatorio.** El supervisor debe emitir un evento formal *deadline_missed* por cada contenedor incumplido, permitiendo una evaluación cuantitativa del rendimiento del sistema tras la finalización del ciclo de salida.

4. Diseño del SMA

A continuación se presentan los diagramas resultantes de aplicar esta metodología al proyecto *Warehouse* [5]. Cada diagrama captura una dimensión distinta del sistema y sirve como especificación formal previa a la implementación. Los diagramas de agente y tareas se incluyen individualmente para cada uno de los cinco agentes del sistema, detallando sus creencias iniciales, las percepciones que reciben del entorno y los planes que ejecutan.

4.1. Diagrama de entorno

El entorno del sistema, ilustrado en la Figura 4.1, está implementado en `WarehouseArtifact`. En esta iteración, el conjunto de percepciones y acciones se ha ampliado significativamente para dar soporte a la coordinación distribuida y al ciclo de salida. El rol del entorno se ha transformado: ya no se encarga de la computación de rutas ni de la toma de decisiones estratégicas, sino que actúa como un proveedor de información bruta y un validador de la ejecución física de las acciones solicitadas por los agentes, delegando toda la inteligencia a los planes en `AgentSpeak`.

Las percepciones que el entorno emite a los agentes se dividen en información de difusión global (*broadcast*) y notificaciones individuales necesarias para la navegación y el control:

- `container_at_entrance(CId, Type, Weight, W, H)`: emitida por *broadcast* a todos los agentes para permitir que los robots detecten y reclamen la carga de forma autónoma.
- `robot_pos(X, Y)`: percepción individual que refleja la ubicación actual de cada unidad en el grid.
- `nav_target(TX, TY)`: destino de movimiento enviado al agente tras una acción de posicionamiento para que este gestione su propia trayectoria.
- `expansion_free_cell(D, X, Y)`: comunica las celdas disponibles en la zona de clasificación junto con su distancia Manhattan precalculada respecto al robot.
- `shelf_available(ShelfId)` y `shelf_occupancy(ShelfId, Occ)`: informan de manera global sobre la disponibilidad y el nivel de ocupación de las estanterías.

- `picked(CId)` y `stored(CId, ShelfId)`: confirmaciones de éxito en las operaciones de recogida y almacenamiento.
- `error(ErrorType, Data)`: reporta anomalías operacionales, como conflictos de destino o rutas bloqueadas.
- `outbound_full`: aviso emitido específicamente hacia el agente *transport* cuando la zona de salida requiere vaciado.
- `container_exited(CId)`: notificación al *scheduler* confirmando que un contenedor ha sido procesado y retirado del sistema.

Las acciones que los agentes ejecutan sobre el entorno han sido refactorizadas para reflejar la autonomía de la flota y las nuevas mecánicas de expedición:

- **Gestión de carga y coordinación:** `claim_container(CId)` y `unclaim_container(CId)` para el reclamo atómico; `pickup(CId)` y `pickup_from_shelf(CId, ShelfId)` para la manipulación; y `drop_at(ShelfId)`, `drop_in_outbound(CId)` o `drop_in_expansion(CId)` para el depósito.
- **Navegación y posicionamiento:** `move_step(X, Y)` para desplazamientos celda a celda, y acciones de solicitud de coordenadas como `move_to_container`, `move_to_shelf`, `move_to_outbound` y `move_to_expansion`.
- **Control y mantenimiento:** `accept_task(CId)` y `release_task(CId)` para la gestión formal de tareas; `collect_outbound_containers` para la limpieza de la zona de salida por parte del transporte; y `discard_container(CId)` para la eliminación de carga dañada o inalcanzable.

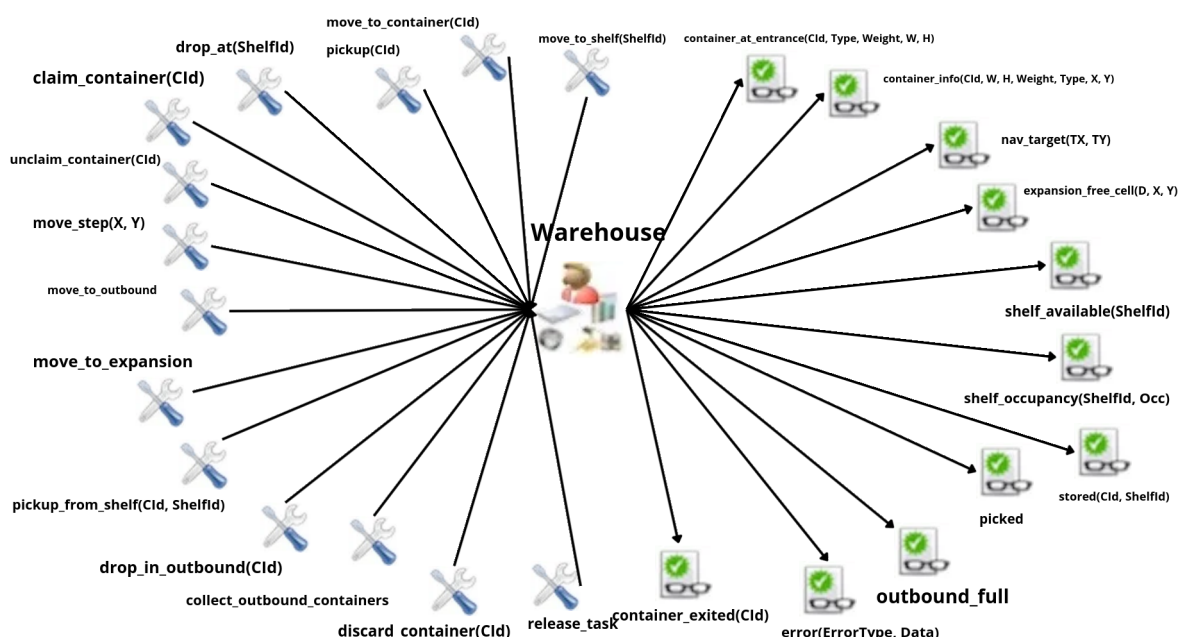


Fig. 4.1. Diagrama de entorno actualizado para la coordinación distribuida.

4.2. Organización del sistema

La organización del sistema experimenta una transformación estructural profunda en esta segunda iteración, transitando desde un modelo jerárquico centralizado hacia una arquitectura de coordinación distribuida y proactiva. El diseño se orienta a maximizar la autonomía de los agentes operativos y a delegar la gestión de estados globales en entidades de control especializadas.

4.2.1. Redefinición de roles y responsabilidades

El cambio fundamental radica en el desplazamiento del centro de gravedad de la toma de decisiones. El *scheduler* deja de ser el centro de coordinación del sistema para asumir un rol acotado exclusivamente a la gestión de los *deadlines* del ciclo de salida. Por su parte, los robots adquieren una autonomía total: perciben la carga directamente a través de señales del entorno, ejecutan el reclamo de contenedores basándose en su propia capacidad y seleccionan la estantería de destino sin necesidad de consultar a ningún otro agente.

4.2.2. Nuevos componentes de la organización

Para dar respuesta a las nuevas exigencias de carga y mejorar la fluidez del almacén, se incorporan dos nuevos agentes al ecosistema organizacional:

- **robot_heavy2:** Se trata de una segunda instancia del robot pesado con capacidades idénticas a `robot_heavy` (capacidad de 100kg y dimensiones de 2x3). Su inclusión es estratégica para gestionar el alto volumen de contenedores pesados. Además, este agente está programado para utilizar con mayor frecuencia el corredor vertical $x = 19$, lo que permite distribuir la carga de tráfico y reducir la congestión en el pasillo central ($x = 9$).
- **Agente transport:** Este agente simula la interfaz logística externa y la llegada de camiones. Es un agente pasivo que reacciona a las señales del `scheduler` al finalizar cada plazo, a la saturación física de la zona de salida o a un temporizador de seguridad de 30 segundos, ejecutando la recogida de contenedores mediante la acción `collect_outbound_containers`.

4.2.3. Evolución del modelo de comunicación

Aunque se mantiene el paso de mensajes (*tell/untell*), el flujo de información ha cambiado radicalmente para eliminar cuellos de botella y dependencias directas:

- **Difusión de estados (Broadcast):** El flujo de asignación centralizada desaparece. El entorno difunde la llegada de carga (`container_at_entrance`) a todos los robots simultáneamente. Del mismo modo, el `scheduler` comunica estados globales como `active_deadline` y `blocked_type` a través de *broadcast*, permitiendo que los robots ajusten su comportamiento de forma independiente.
- **Notificación Asíncrona:** Los robots notifican proactivamente al supervisor sobre las entregas completadas (`container_delivered`) y los errores detectados. Estas notificaciones son de carácter informativo, permitiendo que los robots continúen con su flujo de trabajo sin esperar instrucciones de confirmación o nuevas tareas.

4.3. Diagrama de objetivos ideal

El diagrama de objetivos ideal, representado en la Figura 4.2, describe el comportamiento esperado del sistema bajo una arquitectura distribuida, desglosando la operativa en dos flujos de trabajo principales que conviven de manera independiente: el ciclo de entrada (*inbound*) y el ciclo de salida

(outbound).

El flujo *inbound* se activa de forma reactiva cuando el entorno emite la percepción *container_at_entrance*. A diferencia de la fase anterior, el objetivo principal no es la asignación centralizada, sino el reclamo atómico mediante *try_claim*. Una vez que un robot obtiene el contenedor, asume de forma autónoma el objetivo *select_shelf_and_execute*, que engloba la navegación hacia la carga, su recogida, el transporte hacia la estantería seleccionada y el depósito final. El ciclo concluye con *check_queue*, donde el agente verifica proactivamente si existen más tareas pendientes antes de liberar su estado.

El flujo *outbound* se dispara ante la detección de saturación por parte del supervisor (*storage_full*). Este evento inicia el objetivo *exit_cycle* en el *scheduler*, quien se encarga de gestionar los tiempos de fase y difundir el *active_deadline*. Los robots reaccionan a esta señal activando el objetivo *execute_exit*, que consiste en extraer la mercancía de las estanterías y entregarla en la zona de salida. El proceso finaliza con la intervención del agente *transport*, quien ejecuta la limpieza física del *outbound* tras recibir la petición de transporte.

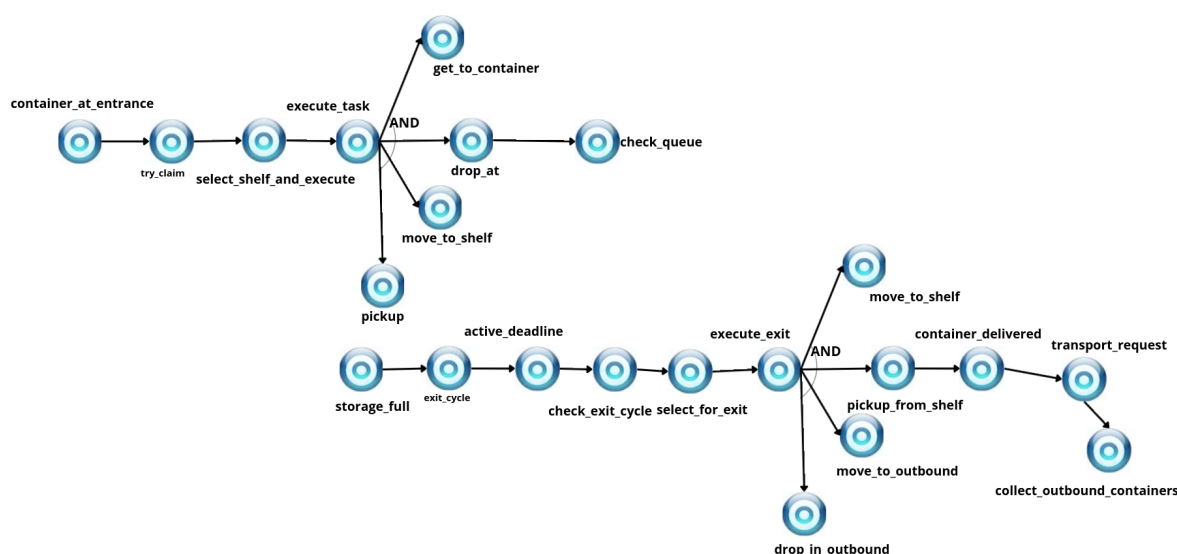


Fig. 4.2. Diagrama de objetivos ideal con flujos inbound y outbound independientes.

4.4. Diagrama de objetivos

El diagrama de objetivos detallado en la Figura 4.3 descompone las metas generales del sistema en sub-objetivos operacionales, reflejando la especialización de cada agente en la segunda iteración.

Esta estructura elimina la dependencia del *get_free_shelf* centralizado y distribuye la lógica de ejecución y control.

Los objetivos de los robots se dividen en tres flujos críticos. El primero, *execute_task*, requiere la compleción de una secuencia bajo un nodo AND que incluye la navegación hacia el contenedor, su recogida, el transporte y el depósito en estantería. El segundo, *execute_exit*, gestiona la logística de salida con una estructura similar pero centrada en la zona *outbound*. Finalmente, el objetivo *safe_expand_drop* actúa como plan de contingencia ante la falta de espacio, dirigiendo la carga hacia la zona de clasificación.

El *scheduler* concentra su capacidad en el objetivo *run_exit_cycle*. Este se bifurca en dos variantes asimétricas: una fase corta para carga urgente y una fase larga para carga estándar y frágil. Cada rama gestiona de forma autónoma la difusión del *deadline* y los tiempos de espera correspondientes, asegurando que el almacén se desbloquee de forma coordinada.

Por último, el supervisor integra objetivos de auditoría y control temporal. Además del ciclo periódico de estadísticas (*stats_loop*), se incorpora el objetivo *monitor_deadline*. Este proceso supervisa de forma proactiva el cumplimiento de los tiempos de entrega, disparando el sub-objetivo *report_deadline_missed* únicamente cuando se detectan contenedores pendientes tras la expiración de una fase.

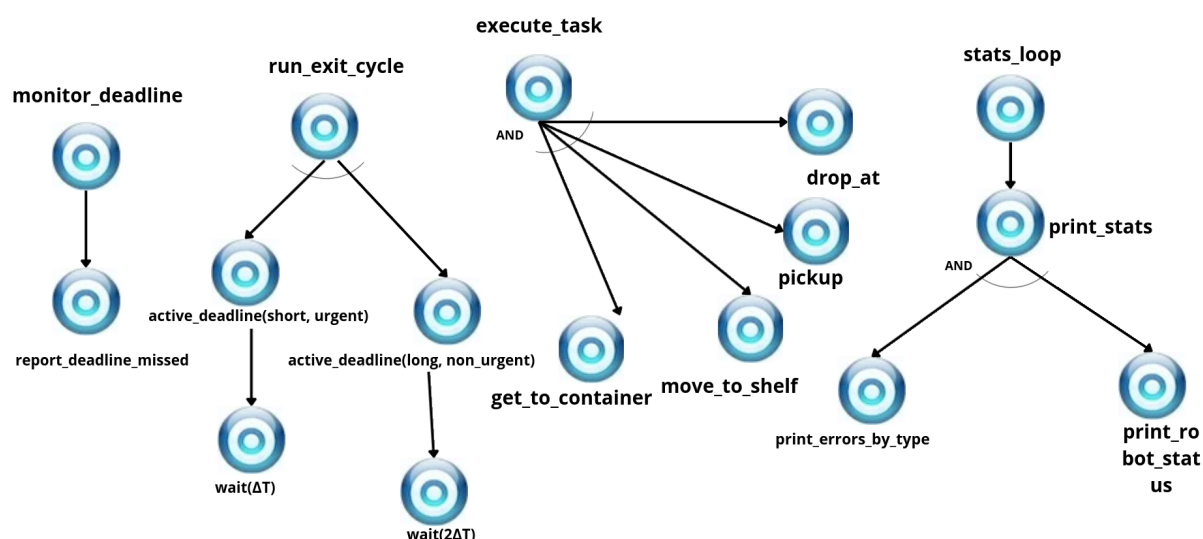


Fig. 4.3. Diagrama de objetivos detallando la descomposición de tareas por agente.

4.5. Diagrama de interacción

El diagrama de interacción de la segunda iteración, representado en la Figura 4.4, refleja una arquitectura descentralizada donde la coordinación surge de la respuesta de los agentes a eventos del entorno y mensajes de difusión, eliminando el flujo de asignación centralizado de la etapa anterior.

En el ciclo de entrada, la interacción comienza cuando el entorno emite la percepción `container_at_entrance` por *broadcast* a todos los robots. Los agentes que cumplen los requisitos de capacidad ejecutan la acción atómica `claim_container` sobre el artefacto Java; el robot que obtiene el reclamo con éxito notifica al supervisor mediante `container_claimed` y procede con el transporte. Al finalizar, el robot informa de la entrega con `container_stored`.

La gestión de la saturación introduce un nuevo flujo de mensajes. Cuando el supervisor detecta que no quedan estanterías disponibles para una categoría, envía `storage_full` al *scheduler*. Este registra el evento y difunde a toda la flota las creencias `blocked_type`, para inhibir nuevas recogidas de ese tipo, y `active_deadline`, para activar el ciclo de salida. Durante esta fase, los robots interactúan con el entorno mediante `pickup_from_shelf` y depositan la carga en la zona de expedición.

El cierre de la fase operativa implica la coordinación con el agente *transport*. Una vez expirado el plazo, el *scheduler* envía `transport_request` al transporte, quien ejecuta la recogida física de

los contenedores acumulados. Finalmente, el *scheduler* retira las creencias de bloqueo y fase activa de los robots mediante un *untell* global.

Adicionalmente, el diagrama incorpora el protocolo de exclusión mutua para el acceso a zonas críticas. Los robots deben solicitar permiso al supervisor mediante *request_zone* y esperar la recepción de *zone_granted* antes de entrar en pasillos o áreas de transferencia, liberando el recurso con *release_zone* al finalizar su tránsito.

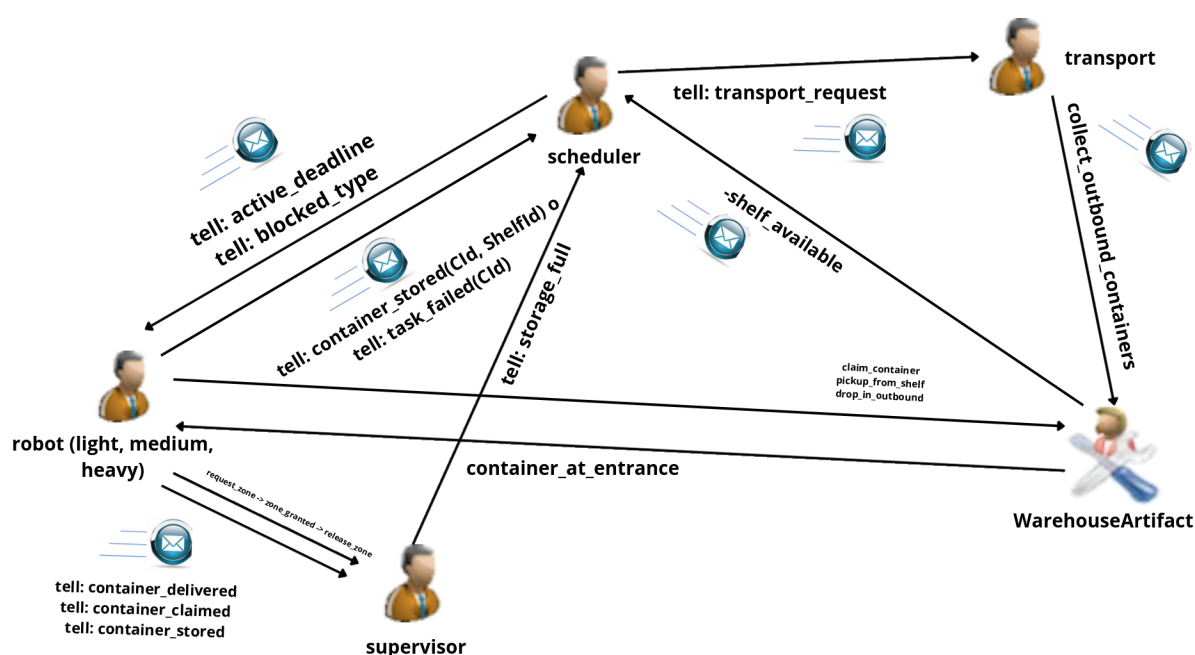


Fig. 4.4. Diagrama de interacción detallando la coordinación distribuida y los protocolos de mutex.

4.6. Agentes y tareas

A continuación se describe la arquitectura cognitiva de cada agente del sistema, incluyendo sus creencias iniciales, las percepciones que reciben del entorno y los planes que ejecutan.

4.6.1. Agente Robot (light, medium, heavy, heavy2)

El diseño interno del agente robot (Figura 4.5) ha sido evolucionado para soportar una arquitectura proactiva y descentralizada. Se han eliminado todas las dependencias de asignación externa, permitiendo que el robot gestione su propio ciclo de vida basado en las percepciones del entorno.

Como se observa en el diagrama, el agente integra cinco creencias de estado (*beliefs*) que definen sus capacidades físicas y su carga actual. El control se organiza en tres grandes intenciones o deseos:

- **execute_task:** Gestiona la entrada de mercancía mediante el reclamo atómico (*claim_container*) y la selección autónoma de estantería.
- **execute_exit:** Responde a los *deadlines* difundidos por el *scheduler* para evacuar carga hacia el *outbound*.
- **safe_expand_drop:** Actúa como protocolo de contingencia cuando las estanterías preferentes están saturadas.

Para que esta autonomía sea posible, el robot procesa ahora percepciones críticas como la ocupación de estanterías (*shelf_occupancy*) y las celdas de expansión libres (*expansion_free_cell*). La interacción física con el almacén se realiza a través de un conjunto de acciones (*actions*) que incluyen la navegación, la carga/descarga y el sistema de reclamo de contenedores.

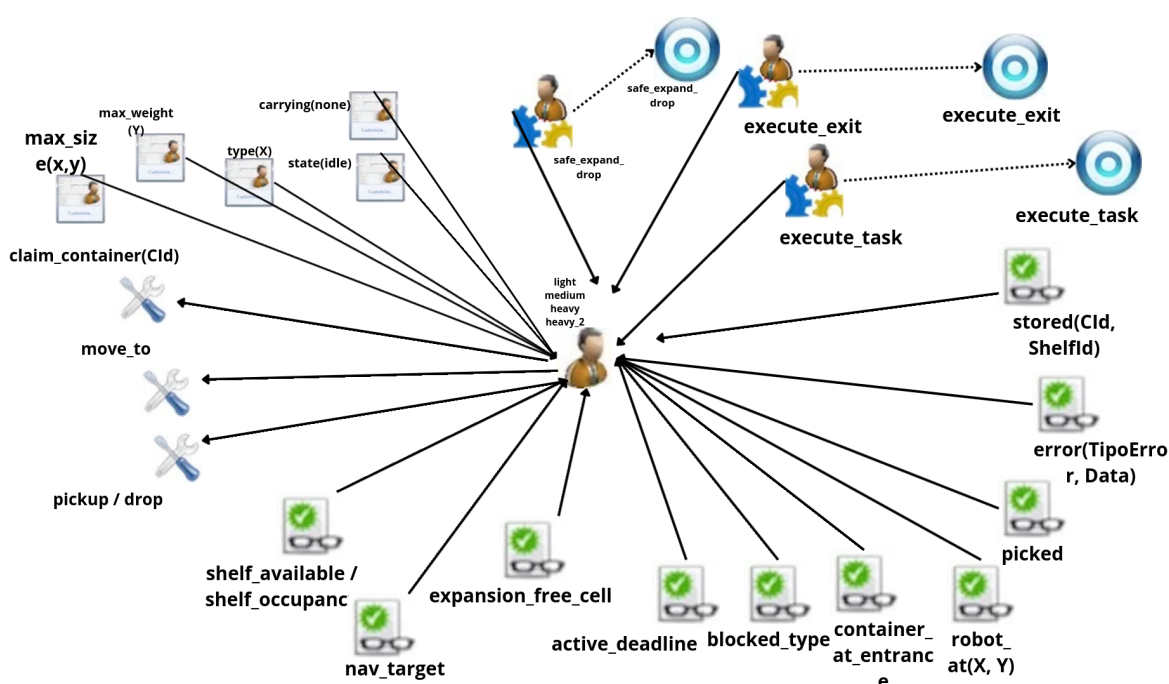


Fig. 4.5. Arquitectura BDI del agente robot autónomo con capacidades de decisión distribuida.

Diagrama de tareas del robot

El diagrama de tareas (Figura 4.6) detalla la lógica algorítmica interna que permite a los robots operar sin una supervisión directa. La principal diferencia respecto a la fase anterior radica en la gestión proactiva de conflictos y la selección de destinos.

El flujo se inicia con la validación del reclamo atómico (`try_claim`). Este mecanismo es crítico en la arquitectura descentralizada, ya que evita que varios robots se desplacen hacia el mismo contenedor. Una vez asegurada la carga, el agente realiza una fase de deliberación interna mediante el plan `pick_shelf`, donde evalúa las creencias de ocupación recibidas por *broadcast* para seleccionar la estantería óptima.

En caso de detectar saturación en las estanterías de su categoría, el diagrama muestra una bifurcación hacia el plan de contingencia `safe_expand_drop`, asegurando que el robot nunca quede bloqueado con carga en sus pinzas. El proceso finaliza con una secuencia coordinada de navegación y manipulación física, tras la cual el agente notifica de forma asíncrona al supervisor para la actualización de las métricas globales del almacén.

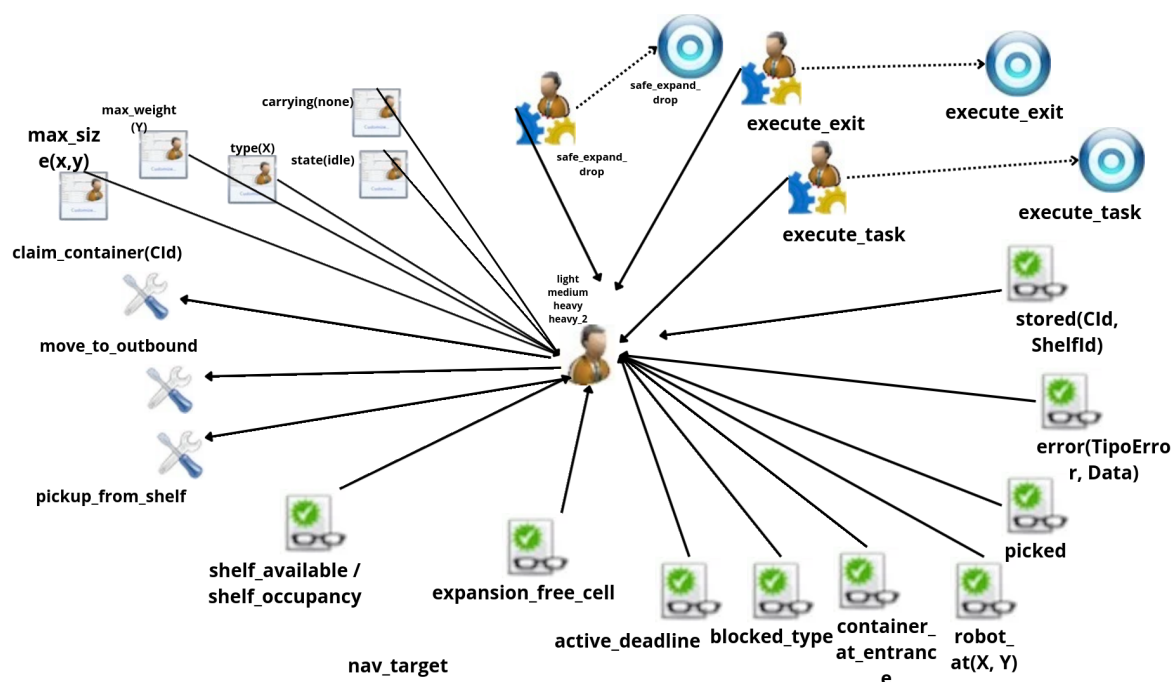


Fig. 4.6. Diagrama de tareas del robot detallando la lógica de reclamo y selección autónoma.

4.6.2. Agente Scheduler

El rol del *scheduler* ha sido redefinido íntegramente, abandonando su función como planificador centralizado para convertirse en el gestor de los estados globales del almacén. Su diseño interno (Figura 4.7) se centra ahora en la coordinación de los ciclos de evacuación y la gestión de la saturación.

El comportamiento del agente es puramente reactivo ante las señales de falta de espacio enviadas por el supervisor (*storage_full*). Al activarse el objetivo *run_exit_cycle*, el *scheduler* razona sobre la urgencia del bloqueo para determinar la duración del *deadline* lógico.

La coordinación con el resto de la flota no se realiza mediante órdenes directas, sino a través de la difusión de creencias globales (*active_deadline* y *blocked_type*). Al finalizar el tiempo de fase, el agente actúa como puente con la logística externa, enviando el mensaje *transport_request* al agente *transport* para asegurar el vaciado físico del área de salida.

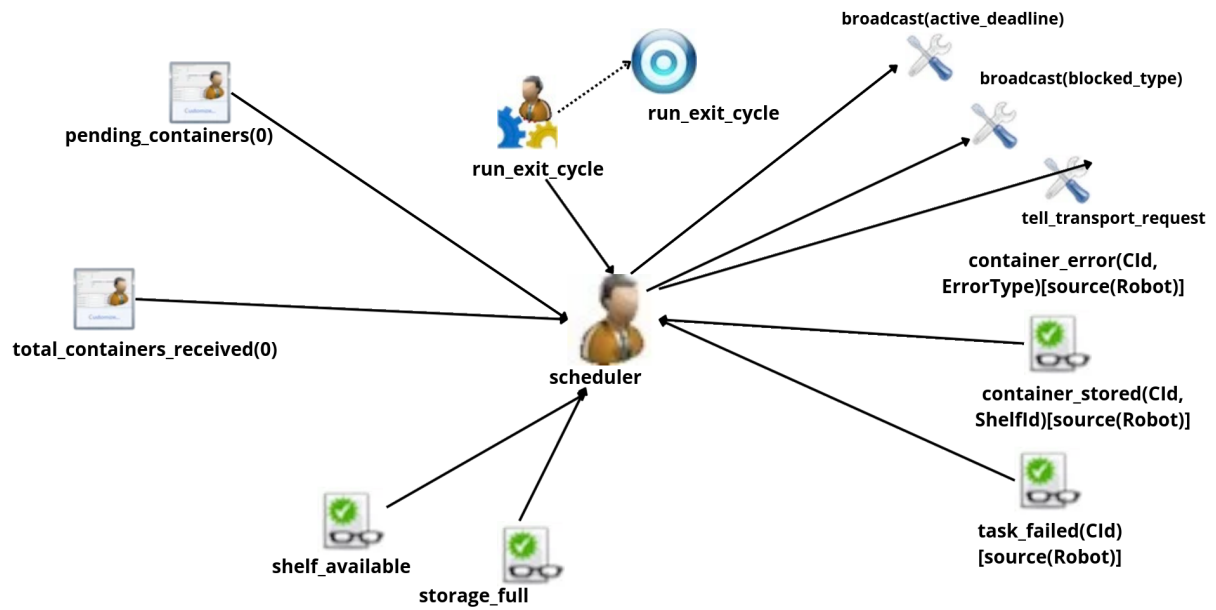


Fig. 4.7. Arquitectura del agente Scheduler centrada en la gestión de ciclos y deadlines.

4.6.3. Agente Supervisor: Arbitraje y Auditoría Crítica

En esta segunda iteración, el rol del supervisor evoluciona de un mero monitor de estadísticas a un **árbitro activo de recursos** y garante de los plazos logísticos. Su arquitectura interna se ha expandido para actuar como el nodo de control que equilibra la autonomía de los robots con las reglas de negocio del almacén.

Funciones Principales y Lógica de Control

El comportamiento del supervisor se desglosa en tres ejes operativos fundamentales que garantizan la estabilidad del sistema:

- **Gestión de Exclusión Mutua (Mutex):** Para garantizar una navegación distribuida segura, el supervisor actúa como gestor de zonas críticas. Los robots, antes de entrar en pasillos o áreas de transferencia, deben emitir un mensaje `request_zone`. El supervisor administra una cola de peticiones y concede el acceso mediante `zone_granted`, asegurando la exclusión mutua en puntos de posible colisión. La liberación del recurso se confirma con la recepción del mensaje `release_zone`.
- **Detección de Saturación y Enlace Logístico:** El agente monitoriza de forma continua la percepción ambiental de disponibilidad de estanterías. En el momento en que detecta que una categoría de carga ha agotado su capacidad, emite de forma proactiva el mensaje `storage_full` dirigido al *scheduler*. Este mecanismo convierte al supervisor en el sensor lógico que dispara el inicio de las fases de evacuación.
- **Auditoría de Cumplimiento (Deadlines):** Durante las fases activas de salida (`active_deadline`), el supervisor inicia un plan de monitorización temporal. Su función es verificar que cada contenedor extraído llegue a la zona *outbound* en el tiempo previsto. Si un agente excede el margen establecido, el supervisor registra un evento de `deadline_missed`, permitiendo una trazabilidad total sobre la eficiencia de la flota.

Finalmente, el agente mantiene su capacidad de **auditoría estadística**, procesando los mensajes `container_stored` y `container_error` para generar informes periódicos (`print_stats`) sobre el rendimiento global del sistema.

4.6.4. Agente Transport: Interfaz de Salida y Seguridad

El agente *transport* es una incorporación crítica diseñada para simular la logística externa de recogida. Su función principal es evitar el colapso físico del almacén, actuando como el único agente capaz de realizar la limpieza definitiva de la zona de expedición mediante la acción `collect_outbound_containers`.

Estrategias de Actuación

Para garantizar la fluidez operativa, el agente opera bajo un modelo de triple redundancia:

- (1) **Recogida por Demanda (Sincronizada):** Al concluir cada ciclo de salida, el *scheduler* emite un mensaje `transport_request`. El transporte reacciona de forma inmediata vaciando la zona *outbound*, lo que permite que el almacén regrese a su estado operativo normal de entrada de mercancía.
- (2) **Recogida Reactiva (Prevención de Bloqueos):** El agente monitoriza la percepción `outbound_full`. Si la zona de salida se satura antes de que expire el plazo oficial, el transporte interviene automáticamente para evitar el *deadlock*, permitiendo que los robots siempre encuentren un hueco libre para depositar la carga.
- (3) **Limpieza Periódica (Seguridad):** Como salvaguarda adicional, el agente ejecuta un ciclo de limpieza cada 30 segundos. Esta estrategia asegura que ningún contenedor quede residualmente en el sistema ante posibles fallos de comunicación o interrupciones en la lógica de los ciclos globales.

El diseño del agente *transport* constituye, por tanto, un componente de seguridad que garantiza que el área de salida sea siempre un recurso disponible para la flota de robots.

5. Arquitectura: Patrón MVC (Modelo-Vista-Controlador)

El entorno del sistema *Warehouse* se fundamenta en el patrón de diseño Modelo-Vista-Controlador (MVC), estructurando una separación estricta entre la representación de datos, la interfaz visual y la lógica operativa. En esta etapa del proyecto, la arquitectura ha evolucionado hacia el principio de **entorno delgado** (*thin environment*): el controlador físico se despoja de cualquier capacidad de decisión estratégica o espacial, limitándose a actuar como una interfaz de datos y motora para los agentes, donde reside ahora toda la capacidad deliberativa.

5.1. El Modelo: datos y estado del almacén

El modelo constituye el núcleo de persistencia del sistema, implementado mediante clases Java en el paquete `src/env/warehouse/`. Los componentes principales (`Container.java`, `Robot.java`, `CellType.java` y `Shelf.java`) encapsulan el estado físico y las invariantes del almacén.

Para garantizar un diseño modular, cada clase asume una responsabilidad específica dentro del dominio logístico:

- `Container.java`: Actúa como el objeto de datos para la carga. Almacena atributos críticos como el identificador, dimensiones, peso, tipo de urgencia y su posición actual en el grid.
- `Robot.java`: Gestiona el estado de cada unidad de transporte, incluyendo su posición, disponibilidad y el vínculo con el contenedor que transporta en cada instante.
- `Shelf.java`: Implementa la lógica de capacidad mediante el método `canStore()`. Esta entidad es responsable de validar si una estantería admite carga adicional evaluando el volumen ocupado y el peso acumulado frente a sus límites físicos.
- `CellType.java`: Define la semántica de la rejilla mediante un enumerado. En esta fase consolidada, el catálogo de celdas incluye tipos base (EMPTY, ENTRANCE, STORAGE, BLOCKED, ROBOT) y las nuevas zonas críticas de la segunda iteración: CLASSIFICATION, destinada a la expansión de carga, y OUTBOUND, para la gestión de expedición.

Una pieza clave en la robustez del modelo es la incorporación de mecanismos de control de concurrencia, destacando la estructura `outboundReservations`. Este `ConcurrentHashMap` resulta fundamental para la coordinación distribuida: permite que un robot realice una reserva atómica de una celda de salida antes de iniciar su ruta. Al centralizar las reservas en el modelo, el sistema garantiza la exclusión mutua en el destino y evita conflictos de ocupación entre múltiples agentes que operan de forma simultánea.

5.2. La Vista: interfaz gráfica

La interfaz gráfica, implementada en la clase `WarehouseView.java` mediante la librería Swing, funciona como un observador pasivo y desacoplado del Modelo. Su responsabilidad se limita a la proyección visual del estado del almacén sin intervenir en la lógica de negocio.

Los elementos clave de la representación incluyen la topografía funcional (identificación de zonas de entrada, salida, expansión y pasillos), el estado dinámico de las estanterías (porcentaje de ocupación en tiempo real) y la monitorización de los agentes, reflejando la posición, tipo de robot y carga transportada mediante códigos de color. El ciclo de refresco es gestionado por el controlador a través del método `repaint()`, lo que garantiza la sincronización constante entre la simulación lógica y la visualización.

5.3. El Controlador: artefacto y agentes

En el patrón MVC de este SMA, el rol del controlador se divide en dos niveles que separan la ejecución física de la deliberación táctica, siguiendo el principio de entorno delgado (*thin environment*). Bajo este esquema, el controlador no toma decisiones estratégicas, sino que expone información, delegando la deliberación espacial íntegramente en los agentes.

El controlador físico (`WarehouseArtifact.java`) actúa como puente entre los agentes y el Modelo. Su función principal es implementar la API de entorno mediante acciones atómicas que validan la física del almacén. Entre las nuevas acciones incorporadas destacan: `claim_container` y `unclaim_container` (para gestionar la propiedad de la carga), `pickup_from_shelf`, `move_to_outbound`, `drop_in_outbound`, `drop_in_expansion`, `collect_outbound_containers` y `discard_container`. Un cambio significativo respecto a versiones anteriores es `move_to_expansion`, que ahora se limita a entregar una lista de celdas libres para que sea el agente quien decida el destino final mediante su propia lógica deliberativa.

Además, el artefacto gestiona la estructura `outboundReservations` mediante un `ConcurrentHashMap`. Este mecanismo es vital para la coordinación distribuida, ya que permite que un robot reserve una celda en la zona de salida de forma atómica. Esto evita condiciones de carrera donde dos agentes podrían dirigirse al mismo punto cuando el *mutex* de la zona se libera antes de que el primer robot haya completado su navegación física.

Los controladores lógicos (ficheros `.asl`) son los agentes del sistema: `scheduler`, `supervisor`, `transport` y la flota de robots. Cada uno implementa su ciclo BDI de forma autónoma: percibe el estado a través de los perceptos que el artefacto le entrega, delibera sobre sus objetivos y emite las acciones correspondientes. En esta arquitectura, los agentes asumen responsabilidades espaciales complejas que anteriormente residían en el entorno, como el cálculo de rutas mediante la heurística Manhattan y la optimización de trayectorias.

El flujo MVC completo se desarrolla de la siguiente manera:

- (1) El artefacto detecta un cambio de estado (ej. saturación de una estantería o llegada de un contenedor) y actualiza el Modelo.
- (2) El artefacto emite los perceptos correspondientes (ej. `storage_full` o `container_at_entrance`) a los agentes.
- (3) Los agentes procesan la información: por ejemplo, el `scheduler` activa el ciclo de salida y los robots deciden autónomamente qué contenedor reclamar mediante la acción `claim_container`.
- (4) El robot navega hacia su destino ejecutando pasos atómicos (`move_step`) que modifican su posición en el Modelo. Si el destino es la zona `OUTBOUND`, el artefacto valida y bloquea la reserva previa en `outboundReservations`.
- (5) La Vista se actualiza tras cada transición de estado, siendo el controlador quien invoca el método `repaint()` para reflejar la situación actual del almacén en la interfaz gráfica.

6. Implementación de la solución

El desarrollo del sistema ha evolucionado desde una arquitectura base hacia un Sistema Multi-agente (SMA) completo, reactivo y robusto. Esta evolución se ha estructurado en dos grandes fases: una primera centrada en la estabilización de la lógica física y el modelo BDI inicial, y una segunda orientada a la descentralización y la gestión compleja de ciclos de salida bajo restricciones temporales.

6.1. El Entorno Java: Interfaz de Percepción y Control

El entorno de simulación actúa como puente entre el plano físico del almacén y la percepción de los agentes. Durante el desarrollo, el entorno ha pasado de contener lógica de decisión a ser un proveedor de primitivas de percepción y ejecución atómica.

6.1.1. Modelo de la cuadrícula y zonas

El almacén se representa como una matriz 2D de 20x15 celdas donde cada posición tiene un tipo lógico definido por el enumerado `CellType`. Los tipos incluyen: `EMPTY`, `ENTRANCE`, `CLASSIFICATION`, `STORAGE`, `SHELF`, `BLOCKED` y `ROBOT`. Esta tipificación permite que los agentes tomen decisiones espaciales sin acceder directamente al estado físico del mapa.

En la segunda iteración, se reorganizó el layout horizontal de las filas $y = 0 - 1$ para incorporar la gestión de expedición:

- **OUTBOUND** ($x = 0 - 2, y = 0 - 1$): Zona de salida, representada en color rojo suave.
- **CLASSIFICATION** ($x = 3 - 4, y = 0 - 1$): Zona teórica de clasificación que se utilizó como zona de expansión, en color amarillo.
- **ENTRANCE** ($x = 5 - 7, y = 0 - 1$): Zona de entrada, en color verde suave.

El desplazamiento de la entrada fue necesario para maximizar el recorrido de los robots en el ciclo de salida, haciendo la simulación más representativa de un entorno industrial real.

6.1.2. Gestión de Contenedores y Ciclo de Vida

Se ha implementado un ciclo de vida estricto para la carga: *spawn* (aparición), *pickup* (recogida), transporte y *drop* (depósito).

- **Generación dinámica:** Los contenedores se generan aleatoriamente en celdas ENTRANCE libres. Si la zona está saturada, el hilo de generación espera hasta que se libere espacio, evitando la pérdida silenciosa de entidades que ocurría en versiones preliminares.
- **Simplificación del *footprint*:** Aunque los contenedores poseen dimensiones reales (W, H) para la clasificación lógica, su representación espacial se simplifica a 1×1 para estandarizar los algoritmos de movimiento.
- **Refactorización de adyacencia:** La lógica para calcular casillas adyacentes se movió a la entidad `Container.java`, permitiendo que los robots se posicionen en la celda perimetral más próxima sin solaparse con la carga.
- **Migración de API deprecada:** Para la generación dinámica de percepciones hacia Jason, se sustituyó `Literal.parseLiteral` por la API moderna `ASSyntax.parseLiteral`.

6.1.3. API de Acciones y Entorno

El entorno `WarehouseArtifact` se diseñó para eliminar la lógica de decisión del código Java, delegándola en los agentes. Las acciones principales incluyen:

- `claim_container(CId)`: Utiliza `ConcurrentHashMap.putIfAbsent` para garantizar un reclamo atómico. La primera llamada tiene éxito y retira el percepto `container_at_entrance` de todos los agentes.
- `unclaim_container(CId)`: Libera el reclamo y re-emite la percepción. Si el robot portaba físicamente el contenedor, lo reposiciona en una celda de entrada libre.
- `pickup_from_shelf(CId, ShelfId)`: Retira carga de la estantería validando la proximidad del robot y restaurando la percepción de disponibilidad (`shelf_available`).
- `move_to_outbound/expansion`: Estas acciones ya no deciden la ruta; emiten percepciones de celdas libres (`nav_target` o `expansion_free_cell`) con distancias Manhattan precalculadas para que el agente elija su destino.
- `drop_in_outbound(CId)`: Deposita el contenedor, libera la reserva en `outboundReservations` y notifica al *scheduler* para la actualización de registros.

6.2. Navegación distribuida en agentes ASL

6.2.1. Por qué se eliminó el BFS

En la primera iteración, la navegación residía completamente en el entorno Java: el método `doMoveTo` calculaba la ruta completa mediante BFS, decidía cada paso de movimiento y manejaba los reintentos ante obstáculos. Técnicamente, el BFS presentaba tres ineficiencias críticas: exploraba hasta 300 nodos del grid en el peor caso, repetía el cálculo completo en cada paso y empleaba una evasión de colisiones estática que invalidaba las rutas si otro robot se cruzaba en el camino. La solución actual delega el movimiento físico al entorno (`move_step`), mientras que la inteligencia de la trayectoria reside íntegramente en los planes ASL del agente.

6.2.2. Heurística *greedy* de distancia Manhattan

El plan `!do_step` aplica una heurística *greedy* de distancia Manhattan: si la diferencia horizontal $|TX - X|$ es mayor o igual que la vertical $|TY - Y|$, el agente intenta avanzar en el eje X; en caso contrario, lo hace en el eje Y. Esta heurística es admisible, ya que nunca sobreestima el coste real en un grid ortogonal, produciendo trayectorias en escalera diagonal que convergen al destino con el mínimo de pasos posible. Si el eje preferido está bloqueado, el sistema utiliza el eje alternativo como *fallback*.

6.2.3. Sistema de pasillos verticales

Dado que las estanterías forman bloques compactos (filas $y = 2 - 3$, $y = 6 - 7$ e $y = 10 - 12$), se han implementado dos columnas siempre libres como "autopistas verticales": la columna $x = 9$ (pasillo izquierdo) y la columna $x = 19$ (pasillo derecho). Las reglas de enrutamiento determinan el uso de estos pasillos según el destino:

- **Hacia zona izquierda** ($TX < 9$): El robot sale al pasillo $x = 9$ antes de realizar el desplazamiento vertical.
- **Hacia zona de salida** ($TX \geq 17$): Se utiliza obligatoriamente el pasillo $x = 19$ para evitar bloqueos con las estanterías finales.
- **Cambio de fila interno**: Se emplea el pasillo más cercano ($x = 9$ si $X \leq 14$; $x = 19$ si $X > 14$) para distribuir la carga de tráfico.

6.2.4. Gestión de colisiones con *backoff* escalonado

Cuando `move_step` falla por la presencia de otro robot, el plan `!step_with_retry` gestiona la colisión mediante un contador de bloqueos (*BC*) que escala la respuesta:

- **BC 0-1:** Bloqueo transitorio; espera aleatoria entre 300 y 1500 ms.
- **BC 2-3:** El bloqueo persiste; se ejecuta `!path_backoff`, realizando un paso perpendicular al eje bloqueado para facilitar el esquivar.
- **BC 4-5:** Espera extendida (1500-3000 ms) para permitir que el sistema se despeje.
- **BC 6:** Notificación de `path_blocked` al supervisor y fallo de la intención para resetear el estado del agente.

6.2.5. `nav_limit` y `nav_target`

Se ha implementado `nav_limit(300)` como un *timeout* determinista: si un robot supera los 300 pasos sin alcanzar su objetivo, la intención falla. Este mecanismo sustituye a la antigua creencia `?nav_abort_signal`, eliminando dependencias ocultas en la base de creencias que podían causar fallos silenciosos.

Asimismo, el subobjetivo `!get_to_container(CId, 3)` permite hasta tres reintentos de navegación, recomputando el `nav_target` mediante `move_to_container` en cada ciclo. Esto es vital para corregir situaciones donde la celda adyacente que estaba libre en el momento inicial queda ocupada por un nuevo contenedor antes de que el robot llegue a su destino.

6.3. Coordinación Distribuida y Planificación

La arquitectura del sistema ha transitado desde un modelo de asignación centralizada hacia un esquema de planificación distribuida. Esta decisión responde a los requerimientos de autonomía de la segunda iteración, que prohíben que el *scheduler* asigne tareas o robots de forma directa.

6.3.1. Planificación distribuida como alternativa al planificador central

La primera iteración centralizaba la asignación en el *scheduler*, que recibía los contenedores del entorno, seleccionaba el robot adecuado y le enviaba la tarea. Los objetivos actuales imponen

que el `scheduler` no asigne ni tareas ni robots. Un planificador centralizado clásico, en el sentido de Russell y Norvig [7], generaría asignaciones óptimas del tipo `assign(Robot, CId, ShelfId)` considerando todos los contenedores pendientes y la disponibilidad de los robots. Ese modelo queda descartado por los propios objetivos del diseño.

La solución descentraliza la planificación: cada robot aplica localmente la misma lógica que el planificador global aplicaría. La precondition de exclusión mutua del esquema clásico (robot libre y contenedor disponible) se evalúa de forma distribuida: cada robot comprueba su propio estado y la disponibilidad del contenedor mediante `claim_container`. La selección de estantería replica el criterio *greedy* del planificador: menor ocupación dentro de la categoría compatible. La consistencia global se garantiza mediante el entorno compartido y las primitivas atómicas, no mediante un árbitro central.

6.3.2. Reclamo atómico de contenedores

El ciclo de trabajo de cada robot se activa de dos formas complementarias. La reactiva: el plan `+container_at_entrance` dispara inmediatamente cuando el entorno emite un contenedor y el robot está libre y cuenta con la capacidad para manejarlo. La de *polling*: `!check_pending_containers` en `!work_cycle` recupera contenedores que llegaron mientras el robot estaba ocupado.

En ambos flujos, el robot ejecuta `!try_claim`, que llama a `claim_container` en Java. La implementación con `putIfAbsent` garantiza la exclusión mutua: la primera llamada exitosa retiene el contenedor y las restantes fallan silenciosamente, sin necesidad de mensajes de coordinación entre robots.

Tras un reclamo exitoso, el robot envía `container_claimed(CId, Type, Weight)` al supervisor. Este mensaje es necesario porque `claim_container` retira `container_at_entrance` de todos los agentes de forma atómica en Java: si el supervisor no ha procesado aún esa percepción, no tendría constancia de la llegada del contenedor. El *handler* `+container_claimed` es la ruta principal para contabilizar contenedores recibidos; `+container_at_entrance` actúa como *fallback* en caso de que el percepto llegue antes que el mensaje.

6.3.3. Selección autónoma de estantería

La selección de estantería se implementa en `common.asl`, compartido por los cuatro robots mediante la instrucción `{ include(common.asl) }`. El plan `!pick_shelf` evalúa las creencias `shelf_available` y `shelf_occupancy` para seleccionar la estantería menos cargada de la categoría compatible con las propiedades físicas del contenedor.

La categoría se determina en dos dimensiones: urgencia (*urgent* → S1, S5, S8; *non_urgent* → S2-S9) y tamaño (*light* ≤ 10kg, *medium* ≤ 30kg, *heavy* cualquiera). La urgencia se establece mediante `claimed_type(CId, Type)`, que el robot almacena tras el reclamo. La etiqueta de tipo del contenedor (*urgent*, *standard*, *fragile*) determina la estantería de destino, pero no la identidad del robot que lo recoge, decisión que es puramente de capacidad física.

Si las estanterías de la categoría preferida superan el umbral del 85 % de ocupación, un plan de *fallback* permite usar cualquier estantería con capacidad para el peso específico. Si ninguna puede alojarlo, el robot lo traslada a la zona de clasificación (zona de expansión) mediante `!safe_expand_drop` y notifica `container_in_expansion` al *scheduler*. El plan de expansión utiliza las percepciones `expansion_free_cell` para seleccionar la celda más cercana: la decisión final reside en el agente y no en el entorno.

6.3.4. `blocked_type`: inhibición coordinada de reclamaciones

Cuando el *scheduler* detecta que un tipo de contenedor ha saturado el almacén, difunde `+blocked_type(Type)` a todos los robots. Los planes reactivos y de *polling* incluyen `not blocked_type(Type)` como guardia; así, los robots cesan los intentos de reclamar contenedores de ese tipo sin necesidad de coordinación adicional. Al finalizar el ciclo de salida, el *scheduler* difunde `untell blocked_type`, restaurando la operación normal.

Un robot que falla en el plan `!pick_shelf` definitivamente también añade `+blocked_type(Type)` a su propia base de creencias antes de recibir el *broadcast* del *scheduler*. Esta auto-inhibición previene que el robot re-reclame el mismo tipo en el intervalo entre el fallo y la llegada del mensaje, evitando bucles de *claim*→fallo→*unclaim*.

Asimismo, se emplea el mecanismo `shelf_wait`: cuando un contenedor específico no encuentra ubicación tras los reintentos, `shelf_wait(CId)` previene que sea reclamado durante 20 segundos. Esta inhibición por contenedor permite que el ciclo de salida libere espacio antes de que el contenedor vuelva a competir por recursos.

6.3.5. Mutex de zona y optimización de `check_queue`

El acceso a las zonas críticas (*inbound*, expansión, *outbound*) está regulado por un *mutex* gestionado por el supervisor. Los robots solicitan el acceso con `request_zone`, el supervisor lo concede mediante `zone_granted` y los robots lo liberan con `release_zone`. El supervisor administra las solicitudes mediante una cola, transfiriendo el acceso directamente al siguiente agente sin necesidad de *polling*. El patrón `.wait(zone_granted(Zone))` en Jason suspende la intención hasta que la creencia es recibida.

El *mutex* de la zona *outbound* presenta un comportamiento especial: se libera en cuanto el robot tiene reservada su celda destino (`outboundReservations` en Java), antes de iniciar la navegación hacia ella. Esto permite que varios robots naveguen simultáneamente hacia celdas distintas de salida, maximizando el flujo operativo.

Al concluir cada tarea, `!check_queue` comprueba si existe algún contenedor compatible esperando antes de decidir el retorno a la base. Si hay carga disponible, el robot pasa a `state(idle)` inmediatamente. En caso contrario, emplea el estado intermedio `state(returning)` durante la navegación de retorno; este estado bloquea el trigger reactivo `+container_at_entrance`, evitando que se dispare mientras el robot está en movimiento y genere intenciones paralelas de `move_step`.

6.4. El Agente Scheduler

En la primera iteración, el *scheduler* actuaba como el planificador central del sistema; sin embargo, en la segunda iteración este rol desaparece completamente. El agente ha dejado de asignar tareas o robots en el ciclo *inbound* para centrarse exclusivamente en la gestión del ciclo de salida y sus restricciones temporales (*deadlines*).

6.4.1. Activación del ciclo de salida

El ciclo se inicia cuando el supervisor detecta la ausencia de estanterías disponibles para un tipo de contenedor y envía la señal `storage_full(Type, T0)` al *scheduler*. El agente utiliza la creencia `active_exit_cycle` como un *mutex* para garantizar que solo un *deadline* esté activo en cada instante. Si se recibe un segundo aviso de saturación mientras existe un ciclo en curso, el tipo de contenedor afectado queda bloqueado (`blocked_type`), pero el nuevo ciclo no se activa hasta que el actual finalice de forma ordenada, procesando los tipos pendientes en orden.

6.4.2. Diseño asimétrico de fases

El objetivo `!run_exit_cycle` se implementa mediante dos variantes que responden a la urgencia de la carga saturada:

- **Fase corta:** Para contenedores urgentes, activa `active_deadline(short, urgent, T0)` con una espera de ΔT segundos.
- **Fase larga:** Para contenedores no urgentes, activa `active_deadline(long, non_urgent, T0)` con una espera de $2 \cdot \Delta T$ segundos.

A diferencia del modelo secuencial sugerido en el enunciado ($T_0, T_0 + \Delta T, T_0 + 3 \cdot \Delta T$), el sistema activa únicamente la fase correspondiente al tipo que generó la saturación. Esta decisión responde a que ejecutar la fase urgente cuando la saturación es de tipo no urgente evacuaría estanterías distintas a las del problema sin liberar el espacio necesario. Los objetivos semanales concretos únicamente exigen un máximo de un *deadline* activo simultáneamente, condición garantizada por el *mutex* `active_exit_cycle`.

El valor de $\Delta T = 90$ segundos se estableció para proporcionar margen suficiente a los robots pesados (que requieren entre 20 y 35 segundos por trayecto) y minimizar los casos de `deadline_missed` observados con $\Delta T = 60$ s bajo alta congestión. Asimismo, este valor evita el bloqueo de entrada de 4 minutos que producía un $\Delta T = 120$ s en la fase larga.

6.4.3. Desbloqueo anticipado al recuperar espacio

El *scheduler* monitoriza la recuperación de capacidad mediante la percepción `shelf_available`. Si una estantería del tipo bloqueado queda libre antes de finalizar el tiempo cronológico de la fase, el agente realiza un *untell* de la creencia `blocked_type` a todos los robots. Este mecanismo permite reanudar la entrada de carga en cuanto existe capacidad real, reduciendo los tiempos de inactividad operativa.

6.5. Los Agentes Robot

Los agentes físicos han sido rediseñados para maximizar su autonomía reactiva, operando bajo un modelo de coordinación emergente y compartiendo una arquitectura base común.

6.5.1. Arquitectura compartida mediante `common.asl`

Los cuatro robots (`robot_light`, `robot_medium`, `robot_heavy` y `robot_heavy2`) comparten los planes lógicos más complejos a través del fichero `common.asl`. Este módulo centraliza la lógica de selección autónoma de estantería (`!pick_shelf`), el reclamo atómico de carga, la gestión de la zona de expansión y los protocolos de *mutex* de zona. Esta estructura elimina la redundancia de código y mitiga el riesgo de divergencia en el comportamiento de la flota.

6.5.2. Capacidades diferenciadas y ciclo inbound

Cada perfil de robot evalúa las percepciones de entrada (`container_at_entrance`) mediante guardias de capacidad propias, diferenciando entre carga ligera ($\leq 10\text{kg}$), mediana ($\leq 30\text{kg}$) y pesada (hasta 100kg). Los robots operan en un bucle reactivo complementado con *polling* (`!check_pending_containers`), lo que les permite recuperar tareas que llegaron mientras se encontraban ocupados. Los tiempos de espera entre las fases de ejecución se han ajustado para simular el tiempo de manipulación proporcional al peso y volumen del contenedor.

6.5.3. Ciclo de salida autónomo

Al recibir la creencia `active_deadline` por difusión (*broadcast*), los agentes inician el ciclo `!check_exit_cycle`. Los robots identifican los contenedores almacenados localmente (`stored`) que coinciden con la urgencia activa y que no han sido reclamados previamente (`exit_claimed`). El proceso incluye la recogida desde la estantería (`pickup_from_shelf`) y el transporte hacia el *outbound*. En caso de fallo en la entrega tras haber recogido la carga, se activa un plan de retorno a la estantería de origen o, en última instancia, un *unclaim* que reposiciona físicamente el contenedor en la entrada para que sea accesible por otros agentes.

6.5.4. `nav_failed` y `shelf_wait`

Para garantizar la fluidez del sistema, se han implementado mecanismos de inhibición temporal:

- `nav_failed`: Si un robot falla tres veces al intentar alcanzar un objetivo, añade esta creencia con un *cooldown* de 20 segundos, permitiendo que agentes con mejor posición reclamen la tarea.
- `shelf_wait`: Se activa cuando un contenedor no encuentra estantería tras los reintentos permitidos, inhibiendo su reclamo durante 20 segundos para otorgar margen al ciclo de salida antes de que la carga vuelva a competir por recursos.

6.6. El Agente Supervisor

El supervisor ejerce como torre de control y auditoría, monitorizando eventos y realizando el seguimiento temporal de los *deadlines*.

6.6.1. Detección de saturación

El agente detecta de forma activa la saturación del almacén mediante dos vías: la percepción directa de la retirada de `shelf_available` en el entorno y los reportes de error (`no_shelf_space`) procedentes de los robots. Para determinar con precisión el tipo de carga al notificar, el supervisor consulta su registro histórico de tipos y emplea operaciones de búsqueda sobre los contenedores almacenados en estanterías no urgentes. El supervisor lee directamente la creencia local de estado de los robots, mantenida de forma reactiva, evitando bloqueos por comunicaciones síncronas.

6.6.2. Seguimiento de deadlines e incumplimientos

Al activarse un ciclo de salida, el supervisor inicia un monitor periódico que comprueba cada 5 segundos si se ha superado el tiempo límite frente a los contenedores pendientes en estantería. El criterio de incumplimiento se define por la condición $T_{now} \geq T_0 + \Delta T$ (o $2 \cdot \Delta T$ para no urgentes) junto a la existencia de carga compatible sin entregar. El sistema emplea un mecanismo dual de monitorización periódica y *backup* reactivo al final de la fase para asegurar que cada incumplimiento genere exactamente un evento `deadline_missed`.

6.7. El Agente Transport

El agente *transport* simula la logística de recogida externa mediante tres mecanismos independientes de operación:

- (1) **Por demanda:** Atendiendo a la solicitud `transport_request` del *scheduler* al finalizar cada *deadline*.
- (2) **Reactivo:** Activado ante la percepción de `outbound_full`, permitiendo desbloquear inmediatamente a los robots que esperan para depositar carga.
- (3) **Periódico:** Una recogida de seguridad cada 30 segundos para evitar acumulaciones residuales que no llenaron completamente la zona de salida.

6.8. Robustez y Gestión de Fallos

El sistema garantiza la consistencia operativa frente a excepciones mediante una jerarquía de planes de contingencia:

- **Zona de expansión:** Funciona como válvula de seguridad ante estanterías saturadas. Los robots depositan la carga en celdas de clasificación y el *scheduler* reintenta la asignación tras un tiempo de espera.
- **Gestión de aplastamientos:** El entorno detecta conflictos físicos y emite una percepción global para que los agentes reaccionen descartando el contenedor afectado.
- **Limitaciones identificadas:** Se ha documentado un posible bloqueo en la zona de entrada para carga pequeña o frágil durante ciclos prolongados de salida. Esto ocurre cuando la entrada se satura de carga no reclamada por el bloqueo activo de un tipo, impidiendo físicamente el acceso de los robots a otros contenedores.

6.9. Logging de eventos obligatorios

La trazabilidad del sistema se asegura mediante un formato de registro uniforme en la consola: `EVENT | time=T | agent=X | type=Y | data=Z`. El tiempo T se calcula en segundos desde la medianoche para garantizar precisión cronológica.

Evento	Agente	Momento de emisión
no_space_detected	supervisor	Saturación de un tipo de estantería
output_phase_started	scheduler	Activación del ciclo de salida al recibir <code>storage_full</code>
deadline_started	scheduler	Al iniciar cada fase (<i>short</i> o <i>long</i>)
deadline_ended	scheduler	Al finalizar cada fase
container_delivered	robot_X	Depósito exitoso en la zona <i>outbound</i>
deadline_missed	supervisor	Expiración de plazo sin entrega
transport_dispatched	transport	Ejecución de recogida de carga del <i>outbound</i>

Tabla 6.1. Resumen de eventos obligatorios monitorizados.

7. Pruebas y Resultados

El sistema fue validado mediante ejecuciones controladas de la simulación, observando el comportamiento del SMA a través de la interfaz gráfica y los reportes por consola emitidos por el agente *supervisor*. Al tratarse de un entorno simulado determinista en su lógica pero no en su temporización, las pruebas se diseñaron para cubrir tanto el flujo nominal como los escenarios de fallo más relevantes. La segunda iteración extiende la batería de pruebas con cuatro escenarios adicionales que validan el ciclo de salida, la zona de expansión y el agente *transport*.

7.1. Escenario base: operación nominal

El escenario base valida el ciclo de entrada de contenedores sin saturación de ninguna estantería. Su propósito es confirmar que los robots operan en paralelo sin colisiones, que los contenedores llegan a sus estanterías correctas y que el *supervisor* registra la actividad de forma coherente.

En la primera iteración, los contenedores eran detectados por el *scheduler* a través del percepto `new_container(CId)`. El planificador clasificaba el contenedor, seleccionaba el robot adecuado y le emitía la tarea correspondiente. Los tres robots operaban en paralelo con el mecanismo `activeDestinations` en Java garantizando que dos robots no reservaran simultáneamente la misma celda destino. El *supervisor* confirmaba la operación mediante el reporte periódico cada 30 segundos.

En la segunda iteración el flujo cambia sustancialmente. La percepción `container_at_entrance` se emite por *broadcast* a los cuatro robots simultáneamente. Cada robot que está en estado `idle` y puede manejar el contenedor ejecuta `claim_container`; la operación `putIfAbsent` en Java garantiza que exactamente uno lo obtiene. El *scheduler* no interviene. La selección de estantería la realiza el propio robot con `!pick_shelf`, eligiendo la menos ocupada de la categoría compatible con el peso del contenedor, sin consultar a ningún otro agente.

En ejecuciones sin saturación observadas durante el desarrollo (de entre dos y cinco minutos) el reporte periódico del *supervisor* no registra errores de capacidad (`container_too_heavy`, `shelf_full`). Los cuatro robots utilizan las nueve estanterías según la categoría de urgencia defini-

da en `common.asl` y la heurística de distribución de carga de `!pick_shelf`. Los únicos incidentes registrados son `nav_failed` puntuales, producidos cuando la zona de entrada se sugestionaba brevemente; el `cooldown` de 20 segundos los resuelve de forma autónoma en todos los casos observados: pasado ese tiempo, el robot vuelve a intentar el reclamo si el contenedor sigue disponible.

7.2. Escenario de saturación del almacén

En la primera iteración, la saturación se manifestaba cuando `!assign_shelf` fallaba repetidamente. El planificador ejecutaba hasta tres reintentos con una espera de 5 segundos entre intentos (`shelf_retries`); agotados estos, notificaba al *supervisor* un error de tipo `no_shelf_space` y descartaba el contenedor. El entorno continuaba generando nuevos contenedores en las celdas ENTRANCE; al acumularse sin ser recogidos, los contenedores entrantes eran aplastados por los robots en tránsito o por contenedores solapados, produciéndose una cascada de eventos `container_broken`. El sistema alcanzaba un estado estable de saturación donde los robots permanecían en `idle` y toda la actividad quedaba registrada en los reportes del *supervisor*.

En la segunda iteración, la saturación desencadena un mecanismo adicional: el ciclo de salida. Cuando los robots que intentan `!pick_shelf` no encuentran estantería disponible para un tipo, llevan el contenedor a la zona de expansión ($x = 3 - 4, y = 0 - 1$) mediante `!safe_expand_drop`. El *supervisor* detecta la retracción de la percepción `shelf_available`: si no queda ninguna estantería de esa categoría de urgencia, emite el evento obligatorio `no_space_detected` y envía `storage_full` al *scheduler*. A partir de ese momento, `blocked_type` impide que los robots reclamen nuevos contenedores del tipo saturado, y `active_deadline` activa el ciclo de salida. Los robots que terminan tareas del ciclo *inbound* comprueban si hay un *deadline* activo y, si es así, seleccionan contenedores de las estanterías correspondientes para llevarlos al *outbound*.

7.3. Ciclo de salida urgente

Este escenario valida el ciclo de salida para contenedores urgentes. La ejecución avanza hasta que las estanterías S1, S5 y S8 se saturan y el *supervisor* detecta que no queda ninguna estantería disponible de esa categoría.

Al detectar la saturación, el *supervisor* emite el evento `no_space_detected` y envía `storage_full("urgent")` al *scheduler*. El *scheduler* registra T_0 , difunde `blocked_type("urgent")` a los cuatro robots para detener la recepción de nuevos urgen-

tes, y activa la fase corta con `active_deadline(short, urgent, T0)`. En el log se observa la secuencia:

```
no_space_detected → output_phase_started → deadline_started
→ container_delivered (una línea por entrega) → deadline_ended →
transport_dispatched
```

Durante la fase, los robots seleccionan autónomamente contenedores urgentes de S1, S5 y S8 marcados como no reclamados aún para el ciclo de salida (`not exit_claimed`), los extraen con `pickup_from_shelf`, navegan al *outbound* y depositan con `drop_in_outbound`. La verificación principal de este escenario es que `blocked_type` impide la entrada de nuevos urgentes mientras el ciclo está activo, y que al terminar la fase el *scheduler* retira `blocked_type` y notifica al agente *transport* para que vacíe el *outbound*.

En los escenarios observados con tres o cuatro contenedores urgentes en las estanterías al inicio del ciclo, todos fueron entregados antes de que venciera el plazo de $\Delta T = 90s$. El *supervisor* no registró eventos `deadline_missed` en estas ejecuciones. El valor $\Delta T = 90s$ resulta suficiente incluso en el peor caso observado, congestión de los cuatro robots en los pasillos, donde el transporte desde la estantería más alejada (S9, en $x = 18$) hasta el *outbound* tardó aproximadamente 60 segundos.

7.4. Ciclo de salida no urgente

Este escenario valida el ciclo de salida para contenedores estándar y frágiles. Cuando las estanterías no urgentes (S2-S4, S6-S7, S9) se saturan, el *scheduler* activa `active_deadline(long, non_urgent, T0)` con duración $2 \cdot \Delta T = 180s$ en lugar de la fase corta.

La asimetría del diseño es verificable directamente en este escenario: la saturación de una estantería no urgente no activa la fase urgente. Evacuar S1, S5 y S8 no libera espacio en las estanterías que tienen el problema; ejecutar la fase urgente desperdicia ΔT sin resolver la saturación detectada. Solo se activa la fase larga, que actúa directamente sobre el tipo saturado.

Con los cuatro robots en ciclo de salida durante 180 segundos, el *outbound* puede llenarse en varias ocasiones antes de que termine la fase. Cada vez que `move_to_outbound` no encuentra celda libre, el entorno emite `outbound_full` al agente *transport*, que reacciona inmediatamente con `collect_outbound_containers` sin esperar al fin de la fase. El robot no necesita saber que el camión llegó: simplemente reintenta `move_to_outbound` en el siguiente ciclo y el entorno le devuelve un `nav_target` válido.

Durante este ciclo largo se hace visible la limitación conocida: si el generador del entorno continúa produciendo contenedores frágiles y pequeños ($\leq 10\text{kg}$, 1×1) al mismo ritmo, y la zona de entrada se llena con contenedores *standard* no reclamados (*blocked_type* activo), *robot_light* puede entrar en bucles de *nav_failed* al no poder alcanzar los contenedores pequeños rodeados de *standard*. Las seis celdas de la zona de entrada no absorben la producción de 180 segundos a ritmo constante. Este comportamiento queda registrado en el log como eventos *nav_failed* consecutivos de *robot_light*, pero no produce pérdida de datos: los contenedores permanecen en la zona de entrada hasta que el ciclo termina y *blocked_type* se retira.

7.5. Zona de expansión y desbordamiento de estantería

Este escenario valida el comportamiento del sistema cuando todas las estanterías de una categoría están llenas y un robot no puede depositar su contenedor.

Cuando *drop_at* falla con *shelf_full*, el plan de fallo de *!execute_task* lleva el contenedor a la zona de clasificación ($x = 3 - 4$, $y = 0 - 1$) mediante *!safe_expand_drop*. El entorno emite *expansion_free_cell(D,X,Y)* para cada celda libre de esa zona con la distancia Manhattan precalculada, y el robot selecciona la más cercana mediante *.sort* y *.findall*. La selección de celda la hace el agente, no el entorno: es el único caso donde *move_to_expansion* no devuelve un *nav_target* directo, sino una lista de candidatos para que el agente delibere.

La verificación distingue dos subcasos. Si la zona de expansión tiene celdas libres, el robot deposita con *drop_in_expansion* y notifica *container_in_expansion* al *scheduler (no-op)*. Si la zona de expansión también está llena, el robot reintenta hasta dos veces; si ninguna celda queda libre, ejecuta *discard_container* y libera el reclamo con *unclaim_container*. El contenedor vuelve a emitirse como *container_at_entrance*, esta vez con *shelf_wait* activo para evitar que los robots lo reclamen inmediatamente en bucle. La creencia *shelf_wait* se auto-elimina a los 20 segundos, tiempo suficiente para que el ciclo de salida libere espacio en alguna estantería compatible.

7.6. Outbound lleno: recogida reactiva por Transport

Este escenario valida la reactividad del agente *transport* cuando el *outbound* se llena durante un ciclo de salida con alta actividad.

Con los cuatro robots en ciclo de salida simultáneamente, la zona de *outbound* —seis celdas en

$x = 0 - 2, y = 0 - 1$ — puede llenarse antes de que finalice la fase. Cuando `move_to_outbound` no encuentra ninguna celda libre ni reservada disponible, el entorno emite `outbound_full` al agente *transport*, que reacciona inmediatamente con `collect_outbound_containers` y registra `transport_dispatched` en el log.

La secuencia observable en el log es:

(robot no puede depositar) → `outbound_full` → `transport_dispatched` → `container_delivered` del siguiente robot que reintenta

El robot no espera ninguna señal de que el camión llegó: simplemente reintenta `move_to_outbound` en el siguiente ciclo y el entorno le devuelve un `nav_target` válido. Este mecanismo reactivo es especialmente relevante en ciclos de alta carga donde los cuatro robots podrían llegar al *outbound* en ráfaga: sin él, los robots esperarían al camión periódico de 30 segundos, lo que podría superar el *timeout* de `nav_failed` y producir entregas fallidas registradas como errores.

8. Conclusiones y trabajo futuro

La implementación de este sistema demuestra que el modelo BDI y el lenguaje Jason son herramientas eficaces para desarrollar soluciones logísticas complejas mediante una arquitectura de agentes autónomos y modulares. El avance más significativo ha sido la transición hacia un entorno simplificado, donde se eliminó la lógica pesada de Java (como la navegación BFS) para delegar la inteligencia espacial y la toma de decisiones directamente en los agentes ASL. Gracias a mecanismos de coordinación emergente, como el reclamo atómico de contenedores y la difusión de estados de bloqueo, se ha logrado un sistema descentralizado capaz de mantener la consistencia global y el cumplimiento de plazos temporales sin necesidad de un planificador centralizado, validando así la robustez de la arquitectura ante escenarios de saturación.

En cuanto a posibles mejoras, se identifican líneas de desarrollo que permitirían mitigar las limitaciones observadas en las pruebas de estrés. Sería recomendable explorar mecanismos de enrutamiento que eviten el bloqueo del agente *robot_light* cuando la entrada se satura de carga no reclamada, así como optimizar la selección de contenedores durante el ciclo de salida basándose en la proximidad física al pasillo. Asimismo, la eficiencia del almacén podría mejorar si se permitiera la concurrencia de diferentes tipos de ciclos de salida o si se implementara persistencia de estado para recuperar la trazabilidad de la carga tras un reinicio. Estas propuestas quedan como vías de optimización para potenciar la fiabilidad y el rendimiento del sistema en escenarios industriales de mayor escala.

Bibliografía

- [1] Rao, A. S., & Georgeff, M. P. (1995). BDI agents: From theory to practice. *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*.
- [2] Rao, A. S. (1996). AgentSpeak(L): BDI agents speak out in a logical computable language. *Proceedings of MAAMAW'96* (LNAI 1038). Springer.
- [3] Bordini, R. H., Hübner, J. F., & Wooldridge, M. (2007). *Programming multi-agent systems in AgentSpeak using Jason*. John Wiley & Sons.
- [4] Bordini, R. H., & Hübner, J. F. *Jason: A Java-based interpreter for an extended version of AgentSpeak*. Disponible en: <https://jasonlang.net>
- [5] Padgham, L., & Winikoff, M. (2002). Prometheus: A pragmatic methodology for engineering intelligent agents. *Workshop on Agent-Oriented Methodologies, OOPSLA 2002*.
- [6] Padgham, L., & Winikoff, M. (2004). *Developing intelligent agent systems: A practical guide*. John Wiley & Sons.
- [7] Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.